Relatório Técnico — Projeto Prático de Estrutura de Dados (Tema A: Redes)

FATEC Ribeirão Preto — ADS, 3º semestre · Disciplina: Estrutura de Dados · Docente: Prof. Me. Heber Castro

Autor: Yuri de Souza Marcondes · 12/11/2026

1. Introdução e objetivo

Este trabalho apresenta uma aplicação de terminal em C que cadastra equipamentos de rede em uma árvore binária de busca (BST), liga esses equipamentos por conexões em um grafo e oferece todas as operações por um menu único. Foi escolhido o **tema A — Redes**: cada registro é um equipamento (roteador, switch, computador, ponto de acesso) e cada aresta é uma conexão entre dois equipamentos.

O objetivo é integrar, em um só programa, os TADs estudados na disciplina — BST e grafo como componentes centrais, mais lista encadeada, pilha, fila e um algoritmo de ordenação — com leitura segura de entradas, tratamento de falhas e liberação correta da memória.

O código-fonte está no repositório Git: **https://github.com/YuriMarcondes20/projeto-estrutura-dados**.

2. Modelagem e campos

Todo equipamento é representado pela struct `Registro`, definida em `registro.h`. O ID é a chave de ligação entre a BST, o grafo e o histórico: nenhuma estrutura guarda ponteiro para nó de outra.

```
#define TAM_TEXTO 80   /* 79 caracteres úteis + o '\0' */

typedef struct {
    int  id;                    /* positivo e único */
    char nome[TAM_TEXTO];
    char tipo[TAM_TEXTO];       /* tipo de equipamento */
    char localizacao[TAM_TEXTO];
} Registro;
```

Campo	Tipo	Regra de validação
`id`	`int`	Inteiro positivo e único; rejeita zero, negativo, letras e overflow

Campo	Tipo	Regra de validação
`nome`	`char[80]`	Aceita espaços; não pode ser vazio nem só espaços; no máximo 79 caracteres
`tipo`	`char[80]`	Mesma regra do nome
`localizacao`	`char[80]`	Mesma regra do nome

Leitura segura. Os textos são lidos com `fgets` e os números com `strtol` (módulo `entrada.c`). Quando a linha passa do limite, a entrada é rejeitada e o restante da linha é descartado do buffer antes de pedir de novo, para não contaminar a leitura seguinte.

Organização em módulos. Cada TAD tem uma interface (`include/*.h`) e uma implementação (`src/*.c`). As dependências são poucas: o grafo usa a lista e a fila na BFS e as demais estruturas não se conhecem; a exceção é `sistema.c`, que combina BST, grafo e pilha, e `main.c`, que apenas apresenta o menu e chama o sistema.

3. Justificativa das estruturas

Cada estrutura foi escolhida pelo problema que resolve no sistema, e não apenas por ser exigida.

Estrutura	Problema no sistema	Por que foi escolhida
BST (chave: ID)	Achar, inserir e remover equipamentos pelo ID	Em média O(log n), contra O(n) de um vetor não ordenado; o percurso em-ordem já entrega a listagem em ordem crescente de ID, sem ordenar
Grafo (matriz de adjacência)	Representar ligações entre equipamentos	As conexões formam uma rede com ciclos, que não cabe em árvore. Com no máximo 100 vértices a matriz ocupa 10 mil bytes e responde “existe a aresta A-B?” em O(1)
Mapeamento ID → índice	IDs arbitrários (ex.: 5000) não servem de índice	Os vetores `ids[]` e `usado[]` guardam qual ID ocupa cada posição; posições liberadas são reaproveitadas

Estrutura	Problema no sistema	Por que foi escolhida
Lista encadeada	Guardar os alcançáveis da BFS sem saber quantos são	Cresce conforme a necessidade; com ponteiro `fim`, a inserção no fim custa O(1) e preserva a ordem de descoberta
Fila	Controlar a BFS	FIFO: quem foi descoberto primeiro é visitado primeiro, o que garante a visita por camadas
Pilha	Histórico de alterações	LIFO: o topo é sempre a ação mais recente; a consulta das 5 últimas percorre a partir do topo sem remover nada
Insertion Sort	Mostrar equipamentos por nome	Simples, estável e eficiente para vetores pequenos (até 100 itens); o desempate por ID é direto

A BST guarda os dados completos e o grafo guarda apenas ID e ligações. Assim, cada informação tem um único dono e as duas estruturas se relacionam somente pelo ID.

4. BST e casos de remoção

A BST (`bst.c`) guarda os registros completos, ordenados pelo ID: IDs menores vão para a esquerda e maiores para a direita. IDs repetidos são rejeitados.

Operação	Como funciona
Inserir	Desce comparando o ID com o nó atual até achar uma posição vazia e cria o nó ali. Se o ID já existe, devolve `BST_DUPLICADO` e nada muda
Buscar	Faz o mesmo caminho da inserção, sem alterar nada; devolve `NULL` se não achar
Em-ordem	Esquerda, nó, direita. Numa BST isso sai em ordem crescente de ID
Menor / maior	Menor: desce sempre pela esquerda. Maior: desce sempre pela direita

Operação	Como funciona
Copiar para vetor	Usa o percurso em-ordem para copiar os registros (valores, não ponteiros) para o vetor auxiliar da ordenação
Liberar	Percurso pós-ordem: libera os filhos antes do pai

Remoção

A remoção primeiro localiza o nó e depois trata três casos. Como exemplo, a árvore criada pela inserção de 50, 30, 70, 20, 40, 60 e 80:

```
        50
       /  \
     30    70
    /  \  /  \
  20   40 60  80
```

Caso	Situação	O que o código faz	Exemplo
1	Folha (sem filhos)	Libera o nó e o pai passa a apontar para `NULL`	Remover 20
2	Um filho	O pai passa a apontar direto para esse filho e o nó é liberado	Remover 30 (depois do 20, só resta o 40, que ocupa o lugar dele)
3	Dois filhos	Copia para o nó o dado do **sucessor** (o menor ID da subárvore da direita, ou seja, o próximo na ordem crescente) e remove o sucessor, que cai no caso 1 ou 2 por nunca ter filho esquerdo	Remover 50: o 60 vira a raiz

A função recebe `NoBST **` (ponteiro para o ponteiro do pai), o que permite alterar a raiz ou o vínculo do pai sem tratar casos especiais.

Altura e degeneração

Todas as operações percorrem um caminho da raiz até um nó, então o custo depende da **altura h** da árvore. Com inserções em ordem embaralhada, a árvore tende a ficar equilibrada, com h próximo de $\log_2 n$. Se os IDs forem inseridos já em ordem crescente (10,

20, 30, 40…), cada nó vira filho direito do anterior: a árvore **degenera em uma lista**, com h = n, e as operações passam a custar O(n). Como o sistema limita o cadastro a 100 equipamentos, o pior caso são 100 comparações, mas em volumes maiores seria necessário usar uma árvore balanceada (AVL ou rubro-negra).

5. Grafo e BFS

O grafo (`grafo.c`) é **não direcionado e sem peso**, implementado com matriz de adjacência de 100 × 100. Os vértices são identificados pelo mesmo ID da BST.

```
typedef struct {
    int  ids[MAX_VERTICES];      /* ID que ocupa cada posição */
    char usado[MAX_VERTICES];    /* 1 = posição ocupada, 0 = livre */
    char matriz[MAX_VERTICES][MAX_VERTICES];  /* 1 = existe aresta */
    int  qtd;                    /* vértices ativos */
} Grafo;
```

Mapeamento ID → índice. Como o ID pode ser qualquer inteiro positivo, ele não serve de índice da matriz. A função `indice_de` procura o ID em `ids[]` (até 100 posições) e devolve a posição. Ao criar um vértice, usa-se a primeira posição livre; ao remover, a posição é liberada e a **linha e a coluna correspondentes são zeradas**, para que um vértice novo não herde arestas antigas.

Arestas. A conexão (A, B) é a mesma que (B, A), então a matriz é marcada nas duas posições. São rejeitados, com mensagem e sem alterar o estado: laço (A, A), aresta duplicada em qualquer ordem, remoção de aresta inexistente e ID inexistente.

BFS (busca em largura)

A BFS recebe uma origem existente e lista todos os vértices alcançáveis, incluindo a origem, na ordem de descoberta:

1. Marca a origem como visitada ao enfileirá-la e insere o ID dela na lista encadeada.
2. Retira um vértice da fila e percorre seus vizinhos em **ordem crescente de ID** (os vizinhos são ordenados por ID antes da visita, porque a ordem das posições na matriz não segue a dos IDs).
3. Cada vizinho ainda não visitado é marcado, enfileirado e inserido no fim da lista.
4. Repete até a fila esvaziar. Uma origem isolada produz apenas o próprio ID.

Exemplo com as arestas 10-25, 10-40 e 25-60 (o vértice 40 foi ligado antes do 25, para provar que a ordem independe da inserção), partindo do 10:

Passo	Sai da fila	Vizinhos novos (crescente)	Fila depois	Lista (ordem de descoberta)
início	—	—	10	10
1	10	25, 40	25, 40	10 → 25 → 40
2	25	60 (o 10 já foi visitado)	40, 60	10 → 25 → 40 → 60
3	40	nenhum	60	igual
4	60	nenhum	vazia	**10 → 25 → 40 → 60**

Os vértices 90 e 95, ligados só entre si, não aparecem: não há caminho até eles.

Existência de caminho. Roda a BFS a partir de A e verifica se B está entre os alcançáveis. Se A ou B não existem, o sistema informa o erro, e não responde "não há caminho". Não é calculado o menor caminho.

6. Integração: lista, pilha, fila, ordenação e rollback

Lista e fila. São usadas dentro da BFS: a fila (FIFO, encadeada) controla quem será visitado, e a lista encadeada, com inserção no fim em O(1) pelo ponteiro `fim`, guarda os IDs na ordem de descoberta. A lista é liberada pelo menu depois de exibida.

Pilha de histórico. Pilha dinâmica encadeada, limitada apenas pela memória, com `push`, `pop`, `peek` e uma consulta que mostra até as 5 ações mais recentes sem remover nenhuma. Cada item guarda o tipo da ação e o(s) ID(s). Entram **somente** alterações concluídas com sucesso: cadastro, remoção de registro, adição e remoção de conexão. Buscas, listagens, ordenação, consultas e operações rejeitadas não são registradas.

Ordenação. A cada pedido, o sistema aloca um vetor auxiliar, copia os registros da BST (por valor, em ordem de ID), ordena por nome com Insertion Sort usando `strcmp` (diferencia maiúsculas de minúsculas, sem normalizar acentos), desempata por ID crescente, exibe e libera o vetor. A BST não é alterada. Não se usa `qsort`.

Consistência entre BST e grafo (rollback)

O cadastro válido na BST cria automaticamente o vértice com o mesmo ID; não existe opção para criar vértice isolado. O módulo `sistema.c` executa as operações compostas em etapas e desfaz o que já foi feito se uma etapa falhar:

Operação	Etapas	Se uma etapa falhar
Cadastrar	1) insere na BST; 2) cria o vértice; 3) empilha no histórico	Etapa 2 falha (grafo cheio ou falha de alocação): remove o registro da BST. Etapa 3 falha: remove o vértice e o registro. Nunca fica registro sem vértice
101º cadastro	Igual ao cadastro; o grafo está cheio (100 vértices)	A criação do vértice é recusada e a inserção na BST é desfeita: o usuário recebe a mensagem e a BST continua com 100 registros
Remover registro	1) confere se o ID existe; 2) remove o vértice e suas arestas; 3) remove o nó da BST; 4) empilha	ID inexistente: rejeita sem alterar nada. Se o histórico não puder ser atualizado, a remoção já feita é mantida e o usuário é avisado (limitação descrita na seção 9)
Adicionar / remover conexão	1) altera a aresta; 2) empilha	Se o histórico falhar, a aresta é desfeita

Falha simulada. Como é difícil provocar falta de memória de verdade, a struct `Sistema` tem o campo `simular_falha_vertice`. Quando ligado nos testes, a criação do vértice é tratada como falha, e o teste confere que o registro não ficou na BST, que o vértice não existe e que o histórico não mudou.

7. Análise de complexidade

Notação: **n** = registros na BST; **h** = altura da BST; **V** = vértices do grafo (máximo 100); **d** = grau de um vértice.

Tempo

Operação	Melhor caso	Caso médio	Pior caso	Observação
BST: inserir, buscar, remover	O(1) (alvo na raiz)	O(log n)	O(n)	O pior caso ocorre quando a árvore degenera em lista (h = n)
BST: menor / maior	O(1)	O(log n)	O(n)	Desce por um único lado, então custa O(h)

Operação	Melhor caso	Caso médio	Pior caso	Observação
BST: em-ordem e cópia para vetor	O(n)	O(n)	O(n)	Visita cada nó uma vez
Grafo: criar / remover vértice	O(V)	O(V)	O(V)	Busca linear do ID; remover também zera uma linha e uma coluna
Grafo: adicionar, remover, consultar aresta	O(V)	O(V)	O(V)	Dominado pela busca linear dos IDs; o acesso à matriz é O(1)
BFS	$O(V^2)$	$O(V^2)$	$O(V^3)$	Cada vértice varre uma linha da matriz (V^2 no total). Ordenar os vizinhos por ID com Insertion Sort custa $O(d^2)$ por vértice, o que chega a $O(V^3)$ apenas em grafo completo
Existe caminho	igual à BFS	igual à BFS	igual à BFS	Roda a BFS e procura o destino na lista, O(V)
Lista: inserir no fim	O(1)	O(1)	O(1)	Usa o ponteiro `fim`
Fila: enfileirar, desenfileirar	O(1)	O(1)	O(1)	
Pilha: push, pop, peek	O(1)	O(1)	O(1)	Consultar as 5 últimas também é O(1), pois o limite é fixo
Insertion Sort	O(n) (já ordenado)	$O(n^2)$	$O(n^2)$ (invertido)	Estável; com empates por nome, desempata pelo ID
Cadastrar / remover (sistema)	O(h + V)	O(log n + V)	O(n + V)	Soma o custo da BST e do grafo

Para a ordenação completa, o custo total de uma solicitação é a cópia O(n) mais o Insertion Sort. Com no máximo 100 registros, mesmo o pior caso (cerca de 10 mil comparações) é imperceptível, e o algoritmo foi escolhido pela simplicidade.

Espaço

BST: O(n), um nó por registro (mais O(h) de pilha de chamadas nas funções recursivas). Grafo: O(V²) fixo, pois a matriz de 100 × 100 ocupa 10 mil bytes independentemente de quantos vértices estão em uso. BFS: O(V), com vetor de visitados, fila e lista. Pilha de histórico: cresce com o número de ações, limitada apenas pela memória. Ordenação: O(n) de vetor auxiliar, liberado ao final.

A matriz de adjacência gasta espaço fixo e tem consulta de aresta rápida; uma lista de adjacência gastaria O(V + A) e seria melhor para grafos grandes e esparsos. Para o limite de 100 vértices, a matriz é mais simples e suficiente.

8. Testes e resultados

Os testes são reproduzíveis e não usam framework: cada programa de teste imprime `[OK]` ou `[FALHOU]` por verificação, e os cenários do programa completo usam entrada redirecionada (`sistema.exe < arquivo.txt`), com a saída esperada guardada ao lado.

Todos os testes foram executados no computador do autor (Windows, gcc 16.2.0) em 12/11/2026 e deram o resultado esperado. Nos cenários do programa completo, a saída obtida foi comparada com a saída esperada guardada em tests/cenarios, sem diferenças.

Área	Casos verificados	Resultado
BST	inserção; ID duplicado; busca; remoção de folha, de nó com um filho e de nó com dois filhos; em-ordem; menor e maior	Passou
Grafo e BFS	aresta e duplicata invertida; laço; ID inexistente; remoção de vértice com várias arestas; BFS em ordem crescente; ciclo; componente desconectado; caminho; limite de 100 vértices	Passou
Integração	cadastro cria vértice; remoção apaga vértice e arestas; 101º cadastro rejeitado com rollback; falha simulada com rollback; falhas fora do histórico	Passou
Pilha	push, pop, peek; últimas 5 ações sem remover	Passou
Ordenação	empate por ID; maiúsculas e minúsculas	Passou
Entradas inválidas	ID zero, negativo, letras, lixo após o número, overflow, vazio; texto vazio, só espaços, 80 caracteres (rejeitado) e 79 (aceito); opção de menu inválida	Passou

Os comandos para reproduzir cada teste e os cinco cenários completos (`tests/cenarios`) estão descritos no README do repositório.

9. Conclusão e limitações

O sistema cumpre o que foi proposto: cadastra equipamentos em uma BST, cria e remove automaticamente os vértices correspondentes no grafo, permite conectar e desconectar equipamentos, lista os alcançáveis por BFS, informa se há caminho entre dois IDs, mantém um histórico das alterações e ordena os registros por nome sem alterar a árvore. O código está dividido em TADs com interface e implementação separadas, as entradas do usuário são validadas e a memória é liberada ao encerrar.

A principal lição técnica foi a **consistência entre estruturas**: como BST, grafo e histórico guardam informações relacionadas, cada operação composta precisa de uma ordem de etapas e de um plano para desfazer o que já foi feito quando uma etapa falha.

Limitações

- **BST sem balanceamento.** Inserções em ordem crescente degeneram a árvore em lista e levam as operações a O(n). Uma AVL ou rubro-negra resolveria.
- **Capacidade fixa de 100 vértices.** A matriz ocupa espaço fixo, e a conversão ID → índice é uma busca linear. Uma lista de adjacência com tabela hash escalaria melhor.
- **Histórico na remoção.** Se não houver memória para registrar uma remoção no histórico, a remoção já feita é mantida (não é desfeita) e o usuário é avisado. Cadastros e conexões, ao contrário, são desfeitos por completo.
- **Sem persistência.** Os dados existem só enquanto o programa está aberto; não há gravação em arquivo.
- **Falta de memória simulada.** O rollback por falha de alocação é testado por simulação (o campo `simular_falha_vertice`), e não por esgotamento real da memória.

10. Declaração de uso de IA

Foram usados assistentes de IA da Anthropic (Claude) como apoio ao estudo e ao desenvolvimento. Finalidades:

- **Estudo:** explicação dos conceitos de BST (inclusive a remoção), grafo, BFS, lista, pilha e fila, e da leitura segura de entradas em C.
- **Ambiente:** orientação para instalar o compilador, usar o Git e publicar o repositório.
- **Código:** a IA gerou a maior parte do código-fonte (módulos lista, fila, pilha, BST, ordenação, grafo, sistema, entrada e menu) e dos programas de teste, a partir do enunciado do projeto.

- **Relatório:** rascunho deste documento.

O autor compilou o código, executou os testes no próprio computador, fez os commits no repositório, revisou os arquivos e o relatório, e é responsável pelo conteúdo entregue.

11. Referências

- FATEC Ribeirão Preto. **Projeto Prático Individual — Estrutura de Dados** (enunciado). Prof. Me. Heber Castro, 2026.
- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.
- ZIVIANI, N. **Projeto de algoritmos com implementações em Pascal e C**. 3. ed. São Paulo: Cengage Learning, 2010.
- KERNIGHAN, B. W.; RITCHIE, D. M. **C: a linguagem de programação — padrão ANSI**. Rio de Janeiro: Campus, 1990.